MAC-Based Message Authentication

Using HMAC (Hash-based Message Authentication Code) in Python

Subject	Information Security
Activity Type	Design & Simulation
Algorithm	HMAC-SHA256
Language	Python 3.x
Date	18 April 2026

1. Introduction

A Message Authentication Code (MAC) is a short piece of information used to verify the integrity and authenticity of a message. It ensures that a message has not been tampered with during transmission and that it originates from the expected sender who possesses the shared secret key.

HMAC (Hash-based Message Authentication Code) is a specific construction for calculating a MAC using a cryptographic hash function in combination with a secret key. This activity demonstrates HMAC-SHA256 — one of the most widely used and trusted MAC schemes in modern cryptographic protocols such as TLS, JWT, and API authentication.

2. Objectives

- Design and implement a sender script that generates an HMAC-SHA256 MAC for a given message.
- Design and implement a receiver script that verifies the MAC upon receiving the message.

- Demonstrate how MAC ensures data integrity by testing with both an original and a tampered message.
- Understand the security properties of HMAC including resistance to timing attacks via hmac.compare_digest().
- Analyze the difference between authenticated and unauthenticated messages.

3. Theoretical Background

3.1 What is a MAC?

A MAC is computed using a secret key K and a message M, producing a fixed-size tag T = MAC(K, M). The receiver, who also holds K, independently computes the MAC from the received message and compares it to the received tag. If they match, the message is authentic and unmodified.

3.2 HMAC Construction

HMAC is defined by the RFC 2104 standard and is computed as:

```
HMAC(K, M) = H((K XOR opad) || H((K XOR ipad) || M))

Where:
  H    = Underlying hash function (SHA-256 in this activity)
  K    = Secret key (padded/hashed to block size)
  ipad = Inner padding (0x36 repeated to block size)
  opad = Outer padding (0x5C repeated to block size)
  ||   = Concatenation
```

The double hashing with separate paddings prevents both length-extension attacks and related-key attacks, making HMAC highly secure.

3.3 Security Properties

Property	Description
Data Integrity	Any modification to the message invalidates the MAC
Authentication	Only parties with the secret key can generate a valid MAC
Non-repudiation	Sender cannot deny sending if MAC is valid (with shared key)
Replay Protection	Can be combined with timestamps/nonces to prevent replays
Timing-Safe	hmac.compare_digest() prevents timing-based side-channel attacks

4. System Design & Architecture

4.1 Communication Flow

The system consists of two scripts that simulate a sender-receiver communication channel. Both share a secret key agreed upon prior to communication.

```
┌──────────────────────────────────────────────────────────┐
│                   MAC Authentication Flow                 │
└──────────────────────────────────────────────────────────┘

 SENDER                                RECEIVER
 ──────                                ────────
 Message M                             Receives (M, MAC_tag)
    +                                        │
 Secret Key K ──▶ HMAC-SHA256                │
    │                  │                     │
    └──▶ MAC_tag ──────┤ ──▶ Payload ──▶     │
                       │          JSON       │
                                             ▼
                                     Recompute MAC
                                     using same K
                                             │
                              compare_digest(expected, received)
                                             │
                                   ┌─────────┴─────────┐
                                   │ MATCH?            │
                                   ├──────────────────▶│ VALID ✓
                                   │                   │ Authentic
                                   └──────────────────▶│ INVALID ✗
                                                       │ Tampered
```

5. Source Code

5.1 sender.py — MAC Generation Script

5.2 receiver.py — MAC Verification Script

CODE:

```
import hmac

import hashlib

import json
```

```
SECRET_KEY = b"supersecretkey123"


def generate_mac(message: str, key: bytes) -> str:
    """Generate HMAC-SHA256 MAC for the given message."""
    mac = hmac.new(key, message.encode('utf-8'), hashlib.sha256)
    return mac.hexdigest()


def send_message(message: str) -> dict:
    """Simulate sending a message with its MAC."""
    mac_value = generate_mac(message, SECRET_KEY)
    payload = {
        "message": message,
        "mac": mac_value
    }
    print("[+] Sender Name        : Khushi")
    print("[+] Original Message  :", message)
    print("[+] Algorithm         : HMAC-SHA256")
    print("[+] Generated MAC     :", mac_value)
    return payload


if __name__ == "__main__":
    message = "Hello! This is a secure authenticated message from Khushi."
    payload = send_message(message)
    with open("payload_khushi.json", "w") as f:
        json.dump(payload, f)
```

```
    print("[✓] Payload saved to payload_khushi.json")


import hmac
import hashlib
import json

SECRET_KEY = b"supersecretkey123"


def verify_mac(message: str, received_mac: str, key: bytes) -> bool:
    """Verify the MAC of a received message."""
    expected_mac = hmac.new(key, message.encode('utf-8'), hashlib.sha256)
    expected_hex = expected_mac.hexdigest()
    # Use compare_digest to prevent timing attacks
    return hmac.compare_digest(expected_hex, received_mac)


def receive_message(payload: dict, tampered: bool = False):
    """Simulate receiving and verifying a message."""
    message = payload["message"]
    received_mac = payload["mac"]


    print("[+] Receiver Name     : Khushi")


    if tampered:
        message = message + " [TAMPERED]"
```

```
    is_valid = verify_mac(message, received_mac, SECRET_KEY)

    if is_valid:
        print("[✓] MAC VERIFICATION PASSED - Message is AUTHENTIC")
    else:
        print("[✗] MAC VERIFICATION FAILED - Message has been TAMPERED")

    return is_valid

if __name__ == "__main__":
    with open("payload_khushi.json", "r") as f:
        payload = json.load(f)

    print("--- Test 1: Original message ---")
    receive_message(payload, tampered=False)

    print("--- Test 2: Tampered message ---")
    receive_message(payload, tampered=True)
```

output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\ALA>  d:; cd 'd:\ALA'; & 'c:\python314\python.exe' 'c:\Users\KHUSHI\.vscode\extensions\ms-pyth
on.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51779' '--' 'c:\Users\KHUSHI\Desktop\
ala-3.py'
[+] Sender Name        : Khushi
[+] Original Message  : Hello! This is a secure authenticated message from Khushi.
[+] Algorithm         : HMAC-SHA256
[+] Generated MAC     : 152e53efacc8c95f766d1631bd3d079eda4090daed3507ba29043cec47a2bb90
[✓] Payload saved to payload_khushi.json
--- Test 1: Original message ---
[+] Receiver Name     : Khushi
[✓] MAC VERIFICATION PASSED - Message is AUTHENTIC
--- Test 2: Tampered message ---
[+] Receiver Name     : Khushi
[✗] MAC VERIFICATION FAILED - Message has been TAMPERED
PS D:\ALA>
```

7. Analysis & Discussion

7.1 How MAC Ensures Data Integrity

In Test 1, the receiver successfully verified the MAC because the received message was identical to what the sender originally hashed. The HMAC computed by the receiver matched the tag transmitted by the sender — confirming that no modification occurred during transit.

In Test 2, the message was altered by appending "[TAMPERED]" before verification. Even this small modification caused the HMAC-SHA256 output to change completely — a fundamental property of cryptographic hash functions known as the avalanche effect. The verification therefore failed, correctly signaling tampering.

7.2 Why HMAC is Preferred over Plain Hashing

Aspect	Plain Hash (e.g., SHA-256)
Secret Key	No key used
Authentication	Anyone can compute hash
Length Extension	Vulnerable
Forgery Resistance	None
Use Case	Checksums, fingerprints

7.3 Timing Attack Prevention

The receiver uses hmac.compare_digest() instead of the == operator for MAC comparison. A naive == comparison returns early as soon as it finds a mismatch, leaking timing information that attackers can exploit to guess the MAC byte-by-byte. compare_digest() runs in constant time regardless of where the mismatch occurs, fully preventing timing side-channel attacks.

7.4 Limitations & Enhancements

- Replay Attacks: An attacker who captures a valid (message, MAC) pair can retransmit it. Adding timestamps or sequence numbers to the message before computing the MAC mitigates this.
- Key Distribution: The shared secret key must be securely exchanged beforehand. In practice, asymmetric cryptography (e.g., RSA or ECDH) is used for key establishment.
- Key Rotation: Long-term use of the same key increases risk. Periodic key rotation and use of derived keys (e.g., via HKDF) is recommended.
- Digital Signatures: For non-repudiation with third parties, asymmetric digital signatures (ECDSA, RSA-PSS) should be used instead of symmetric MACs.